

10강

다차원 배열



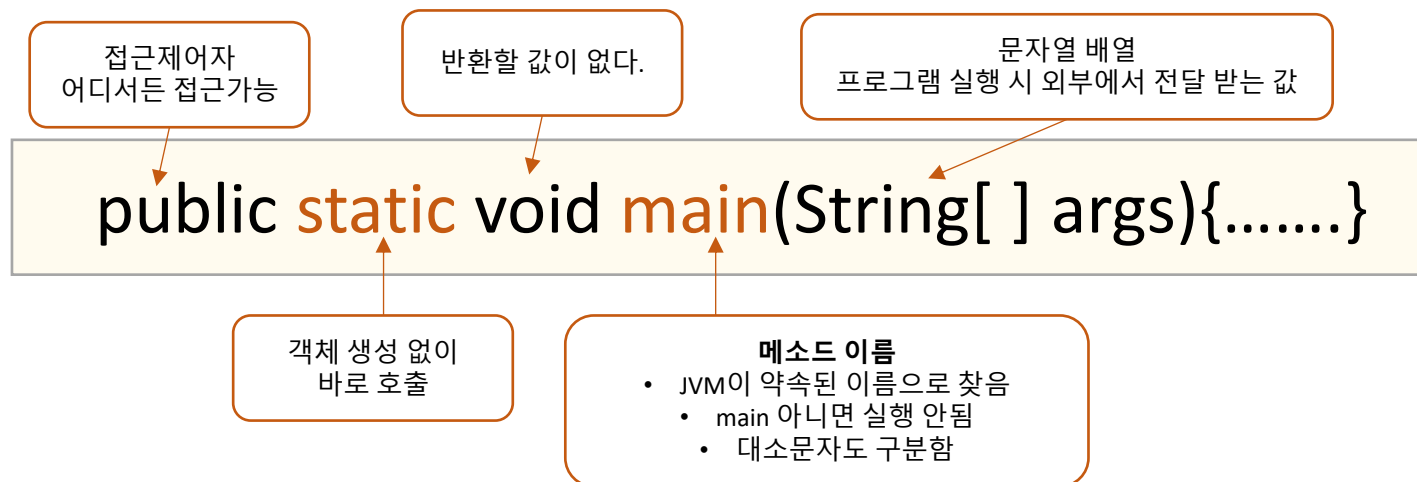
명령 라인





➤ 명령 라인

- 프로그램 실행을 위해서 main()메소드가 필요하다는 것을 이제는 알고 있다. 단, main()메소드의 매개값인 String[] args가 왜 필요한지 궁금증을 해소해 보자
- JVM(Java Virtual Machine)이 **가장 먼저 실행하는 메소드이다.**
- JVM이 객체 생성 없이 어디서든 접근해서 실행할 수 있는 프로그램 시작 메소드이며, **반환 값이 없고** 실행 시 전달된 문자열 데이터를 배열로 받는다
- main(String[] args)의 **args**는 **프로그램 실행 시 외부에서 전달받는 문자열 배열이다.** 사용 여부와 관계없이 main 메소드의 규칙이므로 반드시 작성해야 한다
- JVM은 **public static void main(String[] args)** 모양의 메소드만 실행한다
- 내부 코드에서 args를 쓰든 안 쓰든 선언 자체는 필수이다. 이것이 JVM의 규칙이다.





➤ args를 입력하는 간단한 예제

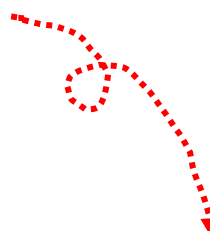
```
package example;

public class Maintest {
    public static void main(String[] args) {
        if(args.length != 2) {
            System.out.println("값의 수가 부족합니다.");
            System.exit(0); //프로그램 강제 종료
        }

        String strNum1 = args[0]; //첫 번째 데이터 얻기
        String strNum2 = args[1]; //두 번째 데이터 얻기

        // Integer.parseInt() => 문자를 정수형으로 변환
        int num1 = Integer.parseInt(strNum1);
        int num2 = Integer.parseInt(strNum2);

        int result = num1 + num2;
        System.out.printf("%d + %d = %d", num1, num2, result);
    }
}
```



Console	Problems	Javadoc
<terminated> Maintest [Java Application] C:\W		
10 + 20 = 30		

💡 args.length를 체크하는 이유

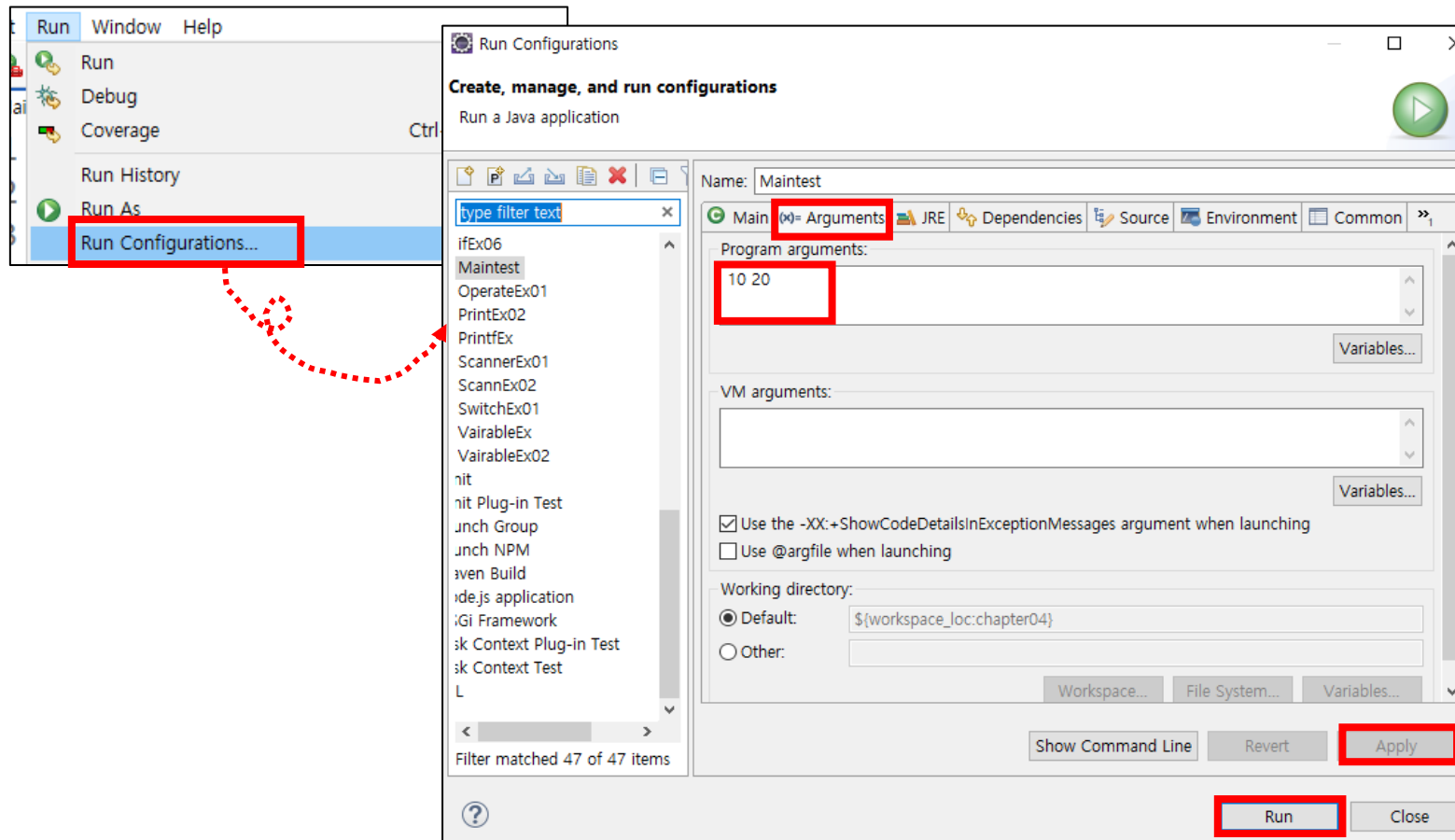
인자가 부족한 상태에서 args[0], args[1]에 접근하면 **IndexOutOfBoundsException(존재하지 않는 index를 사용했다)** 오류가 발생한다. 따라서 미리 개수를 확인하는 것이 안전하다.



➤ args를 입력하는 간단한 예제

💡 중요 포인트

- Run → Run Configurations 메뉴에서 Program arguments에 **10 20**처럼 공백으로 구분하여 입력한다.
- 공백이 포함된 인자를 전달할 때는 **"Hello World"**처럼 따옴표로 감싸야 한다.
- args[0]은 첫 번째 인자(10), arg[1]은 두 번째 인자(20)를 의미한다.





➤ 명령 라인 - String[] args 사용하는 경우

- 웹 / 앱 / 일반 프로그램인 웹 서비스 (Spring, React 등) 사용자 화면이 있는 프로그램, 키보드 입력, 폼 입력, DB 입력이 중심, 입력은 UI, API, DB에서 받는 경우는 사용자가 터미널에서 `java xxx 10 20` 입력하여 사용하지 않는다.
- args는 일반 사용자 프로그램보다는 **서버 실행, 배치 처리, 자동화 작업**에서 사용된다.

실제 사용 예시

1. 서버 실행

`java Server 8080 production`
→ 포트 번호와 환경 설정을 전달

2. 배치 처리

`java DataProcessor input.csv output.csv 1000`
→ 파일 이름과 처리할 데이터 개수를 전달

3. 자동화 작업

`java BackupManager /data/backup daily`
→ 백업 경로와 주기를 전달

4. 테스트 실행

`java TestRunner -verbose -report=summary`
→ 옵션과 설정 값을 전달

 일반 사용자용 GUI 프로그램에서는 args를 거의 사용하지 않지만, 명령줄에서 실행되는 도구나 스크립트, 서버 프로그램 등에서는 필수적으로 사용한다.

다차원 배열





➤ 다차원 배열이란?

- 1차원 배열과 달리 값들이 행들과 열로서 구성된 배열을 2차원 배열이라고 한다. 2차원 배열은 가로 인덱스, 세로 인덱스를 사용한다.
- 자바는 2차원 배열을 중첩 배열 방식으로 구현한다.

실제 사용 예시

- 영화관 좌석 예약 시스템(행, 열)
- 학생 성적표(학생, 과목별 점수)
- 체스판, 바둑판(가로, 세로 좌표)
- 달력(주, 요일)
- 3D 게임 좌표(x, y, z)

1차원 배열

1	2	3	4
---	---	---	---

2차원 배열

1	2	3
4	5	6

3차원 배열

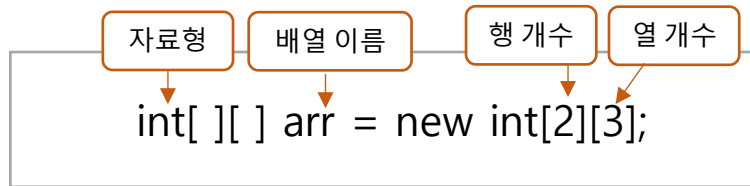
5	6
7	8



➤ 다차원 배열의 선언과 초기화

💡 초기화 방법에 따른 선택

- 값을 **모를 때** → 크기만 지정
- 값을 **아는 경우** → 중괄호로 초기화



// 1. 크기만 지정 (나중에 값 할당)

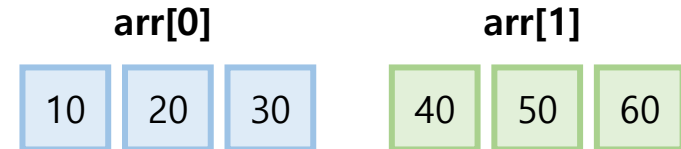
int[][] arr = new int[3][4] ← 초기화 : 0

// 2. 선언과 동시에 초기화

int[][] arr = { {10, 20, 30}, {40, 50, 60} }

// 3. new 키워드와 함께 초기화

int[][] arr = new int[][] { {10, 20}, {30, 40, 50} }





➤ 다차원 배열의 메모리 구조

```
int[ ][ ] arr = new int[3][5]
```

Stack 영역

arr
(참조변수)

- int[] 배열을 3개 담을 수 있는 배열을 만든다
- arr[0] → int[] 를 가리킬 참조 변수
- arr[1] → int[] 를 가리킬 참조 변수
- arr[2] → int[] 를 가리킬 참조 변수
- int 를 담는 게 아니라 int[] 를 가리키는 참조 (reference) 를 담는 배열이다.

Heap 영역

arr (배열 객체)

arr[0]	→
arr[1]	→
arr[2]	→

arr[0] 배열

0	0	0	0	0
---	---	---	---	---

arr[1] 배열

0	0	0	0	0
---	---	---	---	---

arr[2] 배열

0	0	0	0	0
---	---	---	---	---

💡 핵심 포인트

2차원 배열은 실제로 "배열의 배열"입니다. arr는 각 행 배열을 가리키는 참조들의 배열입니다.



➤ 다차원 배열의 메모리 구조

```
int [ ][ ] arr = new int[3][3];
```

```
arr[0][0] = 10;  
arr[0][1] = 20;  
arr[0][2] = 30;
```

```
arr[1][0] = 40;  
arr[1][1] = 50;  
arr[1][2] = 60;
```

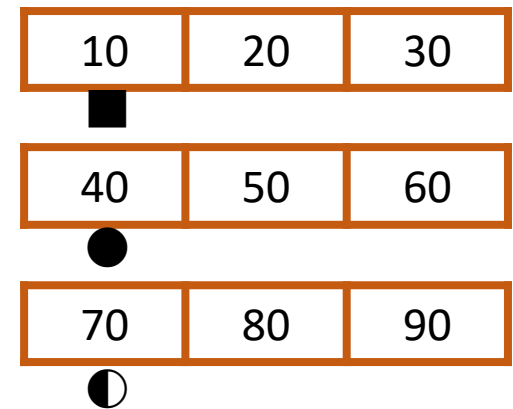
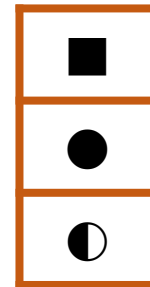
```
arr[2][0] = 70;  
arr[2][1] = 80;  
arr[2][2] = 90;
```

stack

arr



heap





➤ 다차원 배열의 가변 배열

```
int[ ][ ] arr = new int[3][ ]
```

// 각 행의 열개수를 다르게 설정 가능

길이를 정하지 않음

초기 상태

arr[0]

null

arr[1]

null

arr[2]

null

- 2차원 배열은 배열 안에 배열이 들어있는 구조이며, 바깥 배열은 참조를 저장하므로 기본값은 null이다.
- int[] 는 값을 담아서 0으로 초기화되고, int[][] 의 첫 번째 차원은 참조를 담기 때문에 null로 초기화된다.

각 행 생성 후

```
arr[0] = new int[2];
```

0

0

```
arr[1] = new int[4];
```

0

0

0

0

```
arr[2] = new int[3];
```

0

0

0



장점: 메모리 효율성

각 행마다 필요한 만큼만 메모리를 할당할 수 있어 메모리를 절약할 수 있습니다.



➤ 다차원 배열 접근 하기

간단한 예제

```
public static void main(String[] args) {  
    //배열 선언 및 초기화  
    int[][] score= {  
        {95,86},  
        {83,92,96},  
        {78,83,93,87,88}  
    };  
    //배열 접근하기  
    System.out.println(score[0][1]);  
}
```

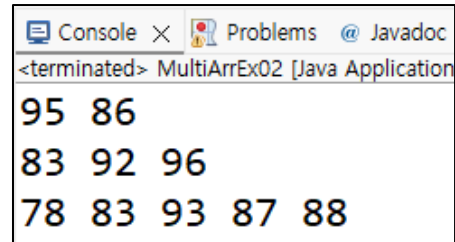
💧 인덱스 이해

Score[0][1]

→ 0번째 행, 1번째 열

전체 순회 하기

```
for(int i=0; i<score.length; i++) {  
    for(int j=0; j<score[i].length; j++) {  
        System.out.print(score[i][j]+" ");  
    }  
    System.out.println();  
}
```



Console × Problems @ Javadoc
<terminated> MultiArrEx02 [Java Application]
95 86
83 92 96
78 83 93 87 88

💡 length 속성 이해하기

- array.length → 행의 개수
- array[i].length → i번째 행의 열 개수

String 객체





➤ String의 equals와 == 연산자 비교

- **== 연산자**
참조 타입에서는 **값**이 아니라 **주소(참조)**를 비교 즉, 같은 객체를 가리키는지를 비교한다.
- **Equals() 메서드**
String 클래스에서 내용(문자열 값)을 비교하도록 오버라이딩하여 문자열이 논리적으로 같은 지 비교한다.
- **결론**
== 연산자는 → 주소 비교, equals() 메소드는 → 내용 비교

```
String a = new String("hongpossible");  
String b = new String("hongpossible");  
  
// a와 b의 물리적 주소 출력  
System.out.println(System.identityHashCode(a));  
System.out.println(System.identityHashCode(b));  
  
if(a==b) {  
    System.out.println("a==b");  
}  
if(a.equals(b)){  
    System.out.println("a.equals(b) // true");  
}
```

```
Console X Problems @ Javadoc  
<terminated> String_example01 [Java Applicatio  
189568618  
793589513  
a.equals(b) // true
```



➤ String의 equals와 == 연산자 비교 문자만

- 문자열 리터럴 "hong" 은 String Pool에 저장된다.
- 같은 문자열 리터럴이 이미 존재하면 새로 만들지 않고 기존 객체를 재사용 한다.
- c와 d는 완전히 같은 객체를 참조하고 있다.
- 주소가 같기 때문에 == 비교도 true가 출력된다.

```
String c = "hong";  
String d = "hong";
```

```
System.out.println(System.identityHashCode(c));  
System.out.println(System.identityHashCode(d));  
  
if(c == d) {  
    System.out.println("c == d");  
}  
if(c.equals(d)) {  
    System.out.println("c.equals(d)는 같다.");  
}
```

```
1922154895  
1922154895  
c == d  
c.equals(d)는 같다.
```




➤ String Pool(문자열 상수 풀)이란?

- String Pool이란 ?
 - ✓ JVM이 문자열 리터럴을 효율적으로 관리하기 위해 미리 만들어 두는 문자열 전용 저장 공간이다.
 - ✓ 중복 문자열을 하나만 생성한다.
 - ✓ 메모리 사용량 감소한다.
 - ✓ 문자열 비교 성능 향상된다.
- String Pool은 어디에 있나?
 - ✓ Heap 메모리 영역 안에 존재하며, 일반 객체 영역과는 논리적으로 분리된 공간이다.
 - ✓ JVM이 자동 관리한다.

String Pool은 문자열 리터럴을 하나만 만들어 공유하기 위한 JVM의 문자열 전용 메모리 공간이다.



➤ String의 equals와 == 연산자 비교

- 문자열 리터럴은 컴파일 시점에 String Pool에 저장된다. 고로 name 은 Pool에 있는 "홍길동" 객체를 참조한다.
- Scanner의 next() 동작 방식은 키보드 입력을 하면 런타임(Runtime)에 결정된다. JVM은 "이 문자열이 상수 인지" 미리 알 수 없기 때문에 String Pool을 사용하지 않고 새로운 String 객체를 Heap에 생성한다.

```
public static void main(String[] args) {  
    // String 객체 배열  
    Scanner scan = new Scanner(System.in);  
    String name = "홍길동";  
    System.out.print("이름을 입력하세요:");  
    String myName = scan.next();  
    System.out.println(myName);  
  
    System.out.println(System.identityHashCode(name));  
    System.out.println(System.identityHashCode(myName));  
  
    if(name == myName) {  
        System.out.println("==연산자 : 일치");  
    }else {  
        System.out.println("==연산자 : 불일치");  
    }  
    if(name.equals(myName)) {  
        System.out.println("equals() : 일치");  
    }else {  
        System.out.println("equals() : 불일치");  
    }  
}
```



```
Console x Problems @ Javadoc  
<terminated> String_example01 [Java Application]  
이름을 입력하세요:홍길동  
홍길동  
189568618  
793589513  
==연산자 : 불일치  
equals() : 일치
```



```
Console x Problems @ Javadoc  
<terminated> String_example01 [Java Application]  
이름을 입력하세요:개나리  
개나리  
==연산자 : 불일치  
equals() : 불일치
```



➤ compareTo() 메서드

- compareTo() 메서드는 String Pool과 전혀 상관없이 **문자열의 내용(문자 코드)만을** 비교한다.
- 2개의 문자열을 비교해 int형 값으로 리턴(반환)해주는 함수이다.

- A.compareTo(B)

- 결과

A와 B가 같으면 0

A가 B보다 사전적으로 순서가 앞이면 음수

A보다 B가 사전적으로 순서가 앞이면 양수

(단, 다른 문자인 경우 해당 문자의 아스키코드의 차이를 출력한다.)



```
String str1 = "가";
String str2 = "나";
String str3 = "가";

// 한글은 유니코드 값: '가' = 44032, '나' = 45208
// 44032 - 45208 = 음수
int rs1 = str1.compareTo(str2);
System.out.println(rs1); // 음수 => -1176

int rs2 = str2.compareTo(str1);
System.out.println(rs2); // 양수 => 1176

int rs3 = str1.compareTo(str3);
System.out.println(rs3); // 0
```

-1176
1176
0

2
-2
-25
0

```
String str4 = "abcd";
String str5 = "ab";
String str6 = "zyxvw";
String str7 = "abcd";

// 영문은 아스키코드 값으로 비교
// "abcd" vs "zyxvw"
// 첫 글자 비교 'a' = 97, 'z' = 122
// 97 - 122 = -25

int rs4 = str4.compareTo(str5);
System.out.println(rs4); // 2

int rs5 = str5.compareTo(str4);
System.out.println(rs5); // -2

int rs6 = str4.compareTo(str6);
System.out.println(rs6); // -25

int rs7 = str4.compareTo(str7);
System.out.println(rs7); // 0
```



Class 명 :String_example03 로 작성 할 것

〈문제〉 사전 순으로 이름 정렬

```
String[] names = {"홍길동", "김유신", "마동석", "자바킹", "서동요"};
```

정답) 김유신, 마동석, 서동요, 자바킹, 홍길동

연습문제





Class 명 :MutiArrayEx01 로 작성 할 것

<문제1> 2차원배열 기본문제[1단계]

```
int[][] arr = new int[3][3];
```

아래 그림과 같이 출력 할 것

10	20	30
40	50	60
70	80	90

<문제2> 2차원배열 기본문제[1단계]

```
int[][] arr = new int[3][3];
```

문제 1) 전체 합 출력

정답 1) 450

<문제3> 2차원배열 기본문제[1단계]

```
int[][] arr = new int[3][3];
```

문제 2) 4의 배수와, 개수를 출력

정답 2) 20 40 60 80

<문제4> 2차원배열 기본문제[1단계]

```
int[][] arr = new int[3][3];
```

문제 3) 4의 배수의 합 출력

정답 3) 200



Class 명 :MutisArrayEx02 로 작성 할 것

<문제5> 2차원배열 기본문제[2단계]

```
int[][] arr = new int[3][3];
```

문제) 인덱스 2개를 입력받아 값 출력

예 1) 인덱스1 입력 : 1
값 출력 : 60

인덱스2 입력 : 2

10	20	30
40	50	60
70	80	90

<문제6> 2차원배열 기본문제[2단계]

```
int[][] arr = new int[3][3];
```

문제) 값 2개를 입력받아 값 교체



Class 명 :MutiArrayEx03 로 작성 할 것

<문제7> 2차원배열 기본문제[3단계]

```
int[][] arr = {  
                {101, 102, 103, 104},  
                {201, 202, 203, 204},  
                {301, 302, 303, 304}  
            };
```

문제 1) 가로 합 출력

정답 1) 410, 810, 1210

문제 2) 세로 합 출력

정답 2) 603, 606, 609, 612



Class 명 :MutiArrayEx04 로 작성 할 것

<문제8>

```
int[][] apt = {  
    {101, 102, 103},  
    {201, 202, 203},  
    {301, 302, 303}  
};  
  
int[][] pay = {  
    {1000, 2100, 1300},  
    {4100, 2000, 1000},  
    {3000, 1600, 800}  
};
```

문제) 각 층별 관리비 합 출력

정답) 4400, 7100, 5400



Class 명 :MutiArrayEx05 로 작성 할 것

<문제9>

```
int[][] apt = {  
    {101, 102, 103},  
    {201, 202, 203},  
    {301, 302, 303}  
};
```

```
int[][] pay = {  
    {1000, 2100, 1300},  
    {4100, 2000, 1000},  
    {3000, 1600, 800}  
};
```

문제) 호를 입력하면 관리비 출력

예) 입력 : 202 관리비 출력 : 2000



Class 명 :MutisArrayEx05 로 작성 할 것

<문제10>

```
int[][] apt = {  
    {101, 102, 103},  
    {201, 202, 203},  
    {301, 302, 303}  
};  
  
int[][] pay = {  
    {1000, 2100, 1300},  
    {4100, 2000, 1000},  
    {3000, 1600, 800}  
};
```

문제) 관리비가 가장 많이 나온 집, 적게 나온 집 출력

정답) 가장 많이 나온 집(201), 가장 적게 나온 집(303)



Class 명 :String_example05 로 작성 할 것

<문제> 쇼핑몰 [관리자]

1. 카테고리과 아이템을 입력 받아 아래의 예시와 같이 저장한다.
2. 카테고리는 각 행의 첫번째 열에 저장한다.
3. 아이템은 각 행의 두번째 열에 저장한다.

단, 아이TEM은 여러개를 추가할 수 있도록 슬러시(/)를 구분자로 연결해준다.

```
Scanner scan = new Scanner(System.in);
String[][] items = new String[100][2];
```

예)

```
{
    {"과일", "사과/포도/"},
    {"과자", "흙런볼/쫄리퐁/"},
    {"음료", "콜라/"},
    {"육류", "소고기/"}
    ...
}
```

< items >

(0,0)	(0,1)
(1,0)	(1,1)
(2,0)	(2,1)
(3,0)	(3,1)
(4,0)	(4,1)

:

(99,0)	(99,1)
--------	--------



[관리자 모드]
 [1]카테고리 관리
 [2]아이템 관리
 [3]전체품목 출력
 [0]종료
 관리자 모드를 선택 :1
 추가할 카테고리 입력 :과일류

[관리자 모드]
 [1]카테고리 관리
 [2]아이템 관리
 [3]전체품목 출력
 [0]종료
 관리자 모드를 선택 :2
 [0] 과일류
 카테고리를 선택하세요:0
 추가할 아이템을 입력하세요:사과

[관리자 모드]
 [1]카테고리 관리
 [2]아이템 관리
 [3]전체품목 출력
 [0]종료
 관리자 모드를 선택 :2
 [0] 과일류
 카테고리를 선택하세요:0
 추가할 아이템을 입력하세요:바나나

[관리자 모드]
 [1]카테고리 관리
 [2]아이템 관리
 [3]전체품목 출력
 [0]종료
 관리자 모드를 선택 :1
 추가할 카테고리 입력 :생선류

[관리자 모드]
 [1]카테고리 관리
 [2]아이템 관리
 [3]전체품목 출력
 [0]종료
 관리자 모드를 선택 :2
 [0] 과일류
 [1] 생선류
 카테고리를 선택하세요:1
 추가할 아이템을 입력하세요:고등어

[관리자 모드]
 [1]카테고리 관리
 [2]아이템 관리
 [3]전체품목 출력
 [0]종료
 관리자 모드를 선택 :

[관리자 모드]
 [1]카테고리 관리
 [2]아이템 관리
 [3]전체품목 출력
 [0]종료
 관리자 모드를 선택 :3
 과일류 : 사과/바나나/
 생선류 : 고등어/

[관리자 모드]
 [1]카테고리 관리
 [2]아이템 관리
 [3]전체품목 출력
 [0]종료
 관리자 모드를 선택 :0
 프로그램 종료



Class 명 :ShoppingMall 로 작성 할 것

<문제> 쇼핑몰 [장바구니]

- 로그인 후 쇼핑 메뉴를 선택하면, 다음과 같이 상품목록을 보여준다.
 - 사과
 - 바나나
 - 딸기
- 번호를 선택해 상품을 장바구니에 담을 수 있다.
- 로그인 회원의 인덱스 번호는 각 행의 첫번째 열에 저장한다.
- 해당 회원이 구매한 상품의 인덱스 번호는 각 행의 두번째 열에 저장한다.

```
String[] ids = {"qwer", "javaking", "abcd"};
String[] pws = {"1111", "2222", "3333"};
String[] items = {"사과", "바나나", "딸기"};
int MAX_SIZE = 100;
int[][] jang = new int[MAX_SIZE][2];
```

예)

{

```
{0, 0},
{1, 1},
{2, 0},
{0, 2},
...
```

}

```
qwer회원
javaking회원
abcd회원
qwer회원
```

```
> 사과구매
> 바나나구매
> 사과구매
> 딸기구매
```

< jang >

(0,0)	(0,1)
(1,0)	(1,1)
(2,0)	(2,1)
(3,0)	(3,1)
(4,0)	(4,1)

:

(99,0)	(99,1)
--------	--------



```

-----
상태 :로그아웃
-----
[GREEN MART]
[1]로그인
[2]로그아웃
[3]쇼핑
[4]장바구니
[0]종료
메뉴 선택 : 1
ID입력 : qwer
PW입력 : 1111
qwer님, 환영합니다.
-----
상태 :qwer
-----
[GREEN MART]
[1]로그인
[2]로그아웃
[3]쇼핑
[4]장바구니
[0]종료
메뉴 선택 : 3

```

```

상품번호를 선택하세요 :2
[상품목록]
[1]사과
[2]바나나
[3]딸기
[4]뒤로가기
상품번호를 선택하세요 :2
[상품목록]
[1]사과
[2]바나나
[3]딸기
[4]뒤로가기
상품번호를 선택하세요 :3
[상품목록]
[1]사과
[2]바나나
[3]딸기
[4]뒤로가기
상품번호를 선택하세요 :1
[상품목록]
[1]사과
[2]바나나
[3]딸기
[4]뒤로가기

```

```

상품번호를 선택하세요 :4
-----
상태 :qwer
-----
[GREEN MART]
[1]로그인
[2]로그아웃
[3]쇼핑
[4]장바구니
[0]종료
메뉴 선택 : 4
--- 내 장바구니 목록 ---
사과: 1개
바나나: 2개
딸기: 1개
-----
상태 :qwer
-----
[GREEN MART]
[1]로그인
[2]로그아웃
[3]쇼핑
[4]장바구니
[0]종료
메뉴 선택 : 0

```